

TEACHER & EDUCATOR MODULES

Complete Technical Architecture & Sequence Diagrams Specification

Document Version:	v2.4.0 (Enterprise Production)
Target Platform:	Web (React/Vite) & Mobile (Android APK/AAB)
Backend API:	Node.js / Express / MongoDB (Render Deployment)
Production API URL:	https://e-learning-63yb.onrender.com/api
Modules Covered:	Dashboard, Course Wizard, Classroom, MCQ Builder, Audit, Payouts
Author / Architecture:	Google DeepMind Agentic Pair Programming

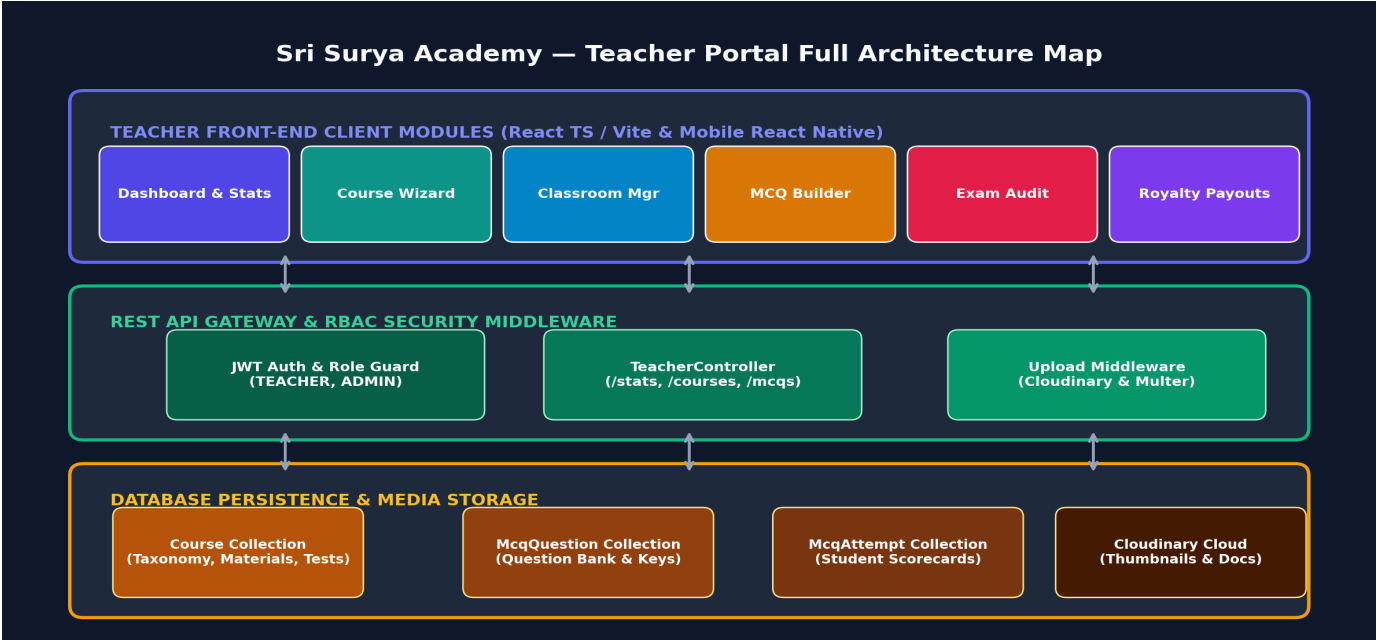
Table of Contents:

- 1. High-Level Teacher System Architecture & Component Mapping
- 2. Module 1: Teacher Authentication & Real-Time Metrics Sequence
- 3. Module 2: 3-Step Course Creation Wizard & Auto-Publish Sequence
- 4. Module 3: Classroom Management & Enrolled Students Tracking Sequence
- 5. Module 4: Topic-Wise MCQ Quiz Builder & Set Creation Sequence
- 6. Module 5: Student Exam Audit & Analytics Sequence
- 7. Module 6: Royalty Earnings & Payout Withdrawal Request Sequence

SYSTEM ARCHITECTURE OVERVIEW

High-Level Teacher System Component Mapping

The Sri Surya Academy Teacher ecosystem comprises three tightly integrated architectural layers: **(1) Educator Client Layer** (Web SPA & React Native Mobile App), **(2) REST Gateway & Security Layer** (Express router guarded by JWT Authentication and Role-Based Access Control enforcing role TEACHER or ADMIN), and **(3) Persistence & Cloud Vault** (MongoDB schemas for Courses, McqQuestions, McqAttempts, and PayoutRequests alongside Cloudinary for multimedia storage).

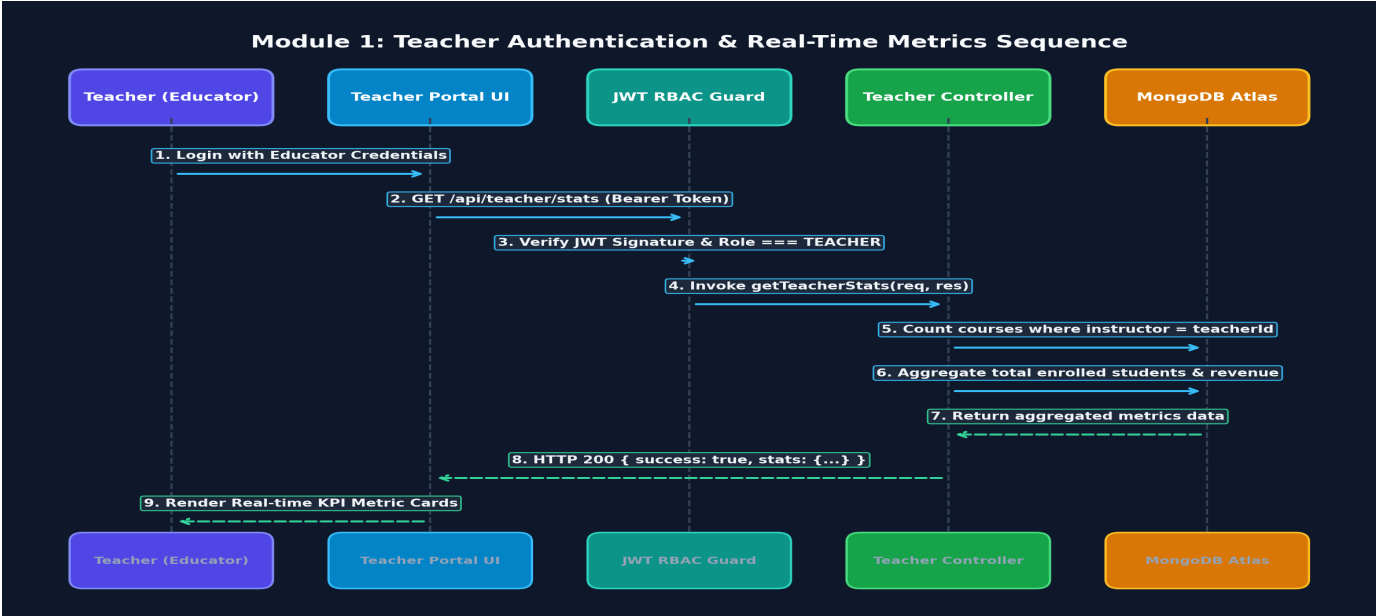


Component	Tech Stack & Role	Security / Policies
Teacher Web Portal	React 18 + Vite, Tailwind/Vanilla CSS, Lucide icons	Protected routes, auto-refresh JWT interceptor
Teacher Mobile APK	React Native 0.87, Android Native bundle, Metro	AsyncStorage secure token vault, deep-links
API Gateway	Node.js / Express.js on Render Cloud	CORS allowlist, Helmet, Rate-limiting
Teacher Controller	teacherController.js (CRUD, aggregates, payout logic)	Strict RBAC guard: restrictTo('TEACHER','ADMIN')
MongoDB Database	MongoDB Atlas (Mongoose ODM schemas)	Compound indexes on category, stateCode, active

MODULE 1 SPECIFICATION

Module 1: Teacher Authentication & Real-Time Metrics Sequence

Upon educator authentication, the portal dispatches an authenticated query to the stats controller. The backend performs non-blocking aggregation over the Course and Enrollment collections, extracting the total courses authored by the educator, total active enrolled learners, accumulated earnings, and live question bank stats.



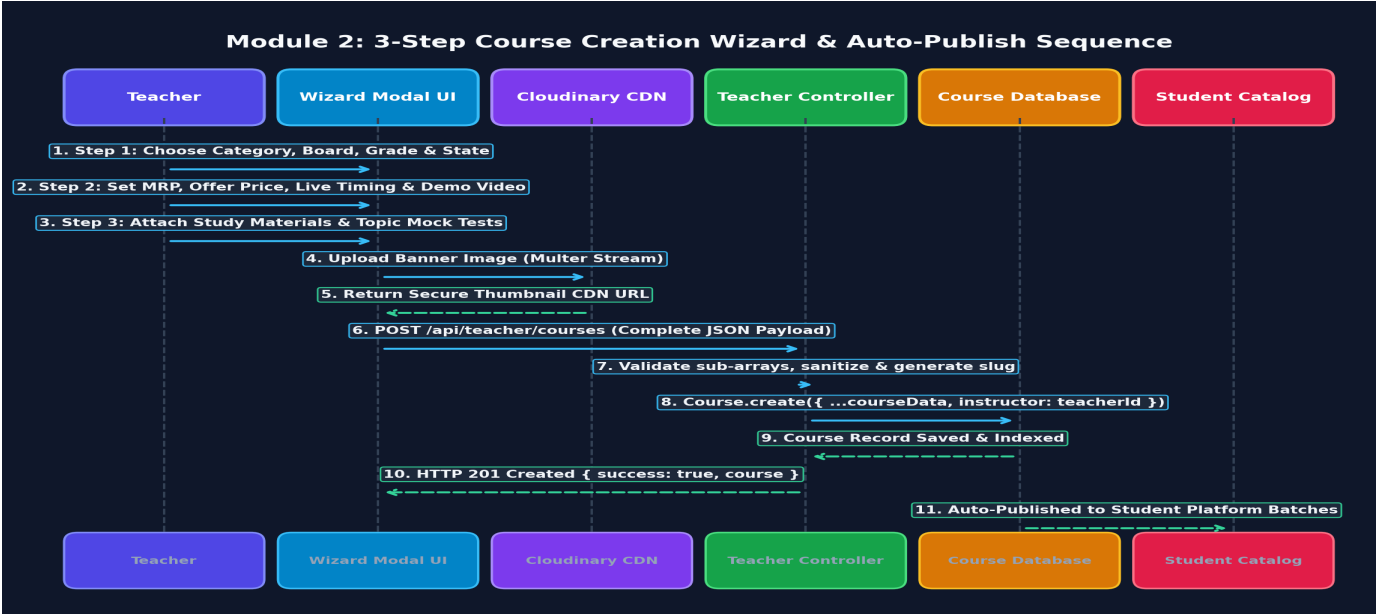
Sequence Execution Steps & Data Exchange:

#	Sender	Receiver	Action / Payload Description	DB / System Impact
1	Teacher	Portal UI	Submits login credentials (email & password)	Client initiates Auth token exchange
2	Portal UI	RBAC Guard	Dispatches GET /api/teacher/stats with Bearer JWT	Transported over HTTPS SSL
3	RBAC Guard	RBAC Guard	Validates JWT signature, expiration & role === 'TEACHER'	Rejects with 401/403 if invalid
4	RBAC Guard	TeacherCtrl	Forwards verified request to getTeacherStats handler	Attaches req.user payload
5	TeacherCtrl	MongoDB	Executes Course.countDocuments({ instructor: teacherId })	Queries active published batches
6	TeacherCtrl	MongoDB	Aggregates enrolled student enrollments and sum of revenues	Calculates net royalty balance
7	MongoDB	TeacherCtrl	Returns aggregated KPI metrics to controller	Low-latency Indexed lookup
8	TeacherCtrl	Portal UI	Returns HTTP 200 { success: true, stats: {...} }	Populates dashboard state
9	Portal UI	Teacher	Renders Active Batches, Enrolled Learners & Revenue Cards	Real-time reactive visual update

MODULE 2 SPECIFICATION

Module 2: 3-Step Course Creation Wizard & Auto-Publish Sequence

The modern 3-step Course Creation Wizard enables teachers to define curriculum structure, pricing models, multi-format study material vaults (PDFs, Notes, E-Books with topic tags), and topic-wise mock tests with embedded MCQ questions. Upon submission, the course is automatically published to the student catalog.



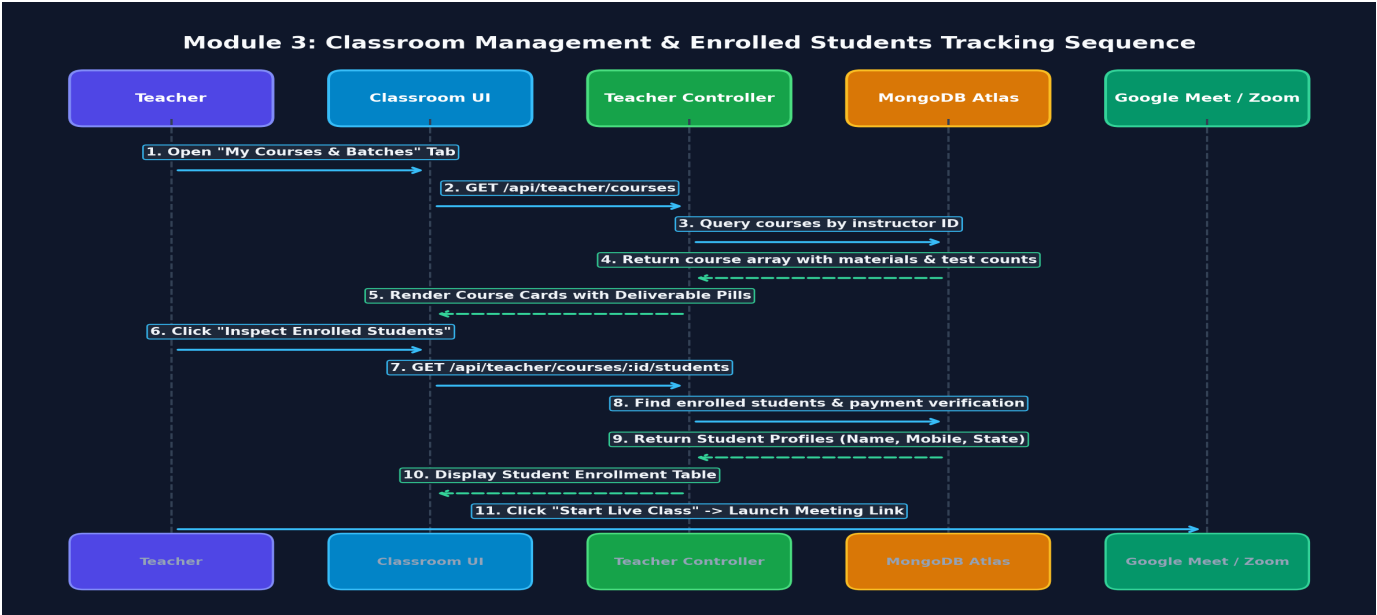
Sequence Execution Steps & Data Exchange:

#	Sender	Receiver	Action / Payload Description	DB / System Impact
1	Teacher	Wizard Modal	Step 1: Selects Category, Grade, Board, Subject & State	Client validates taxonomy node
2	Teacher	Wizard Modal	Step 2: Enters MRP, Offer Price, Validity & Live Schedule	Computes instant discount savings %
3	Teacher	Wizard Modal	Step 3: Attaches Multi-Docs (PDF/DOC) & Topic Mock Tests	Populates studyMaterials & mockTests
4	Wizard Modal	Cloudinary	Streams course banner file via Multer buffer	Multi-region CDN image ingestion
5	Cloudinary	Wizard Modal	Returns secure HTTPS URL and responsive variants	Binds URL to thumbnail payload
6	Wizard Modal	TeacherCtrl	Submits POST /api/teacher/courses with full JSON schema	Authenticated payload dispatch
7	TeacherCtrl	TeacherCtrl	Validates sub-arrays, formats questions, auto-slugifies title	Pre-save schema validation hook
8	TeacherCtrl	Course DB	Executes Course.create({ ...payload, instructor: teacherId })	Document inserted and indexed
9	Course DB	TeacherCtrl	Returns persisted course document with Mongo ObjectId	Database transaction committed
10	TeacherCtrl	Wizard Modal	Returns HTTP 201 Created { success: true, course }	Triggers client notification toast
11	Course DB	Student Catalog	Batch instantly surfaces in Browse Platform Courses catalog	Live student visibility enabled

MODULE 3 SPECIFICATION

Module 3: Classroom Management & Enrolled Students Tracking Sequence

Educators manage their active batches, monitor enrolled students, inspect payment confirmations, and initiate live classroom broadcasts. The module offers deep visibility into student demographic distribution and direct links for scheduled lecture sessions.



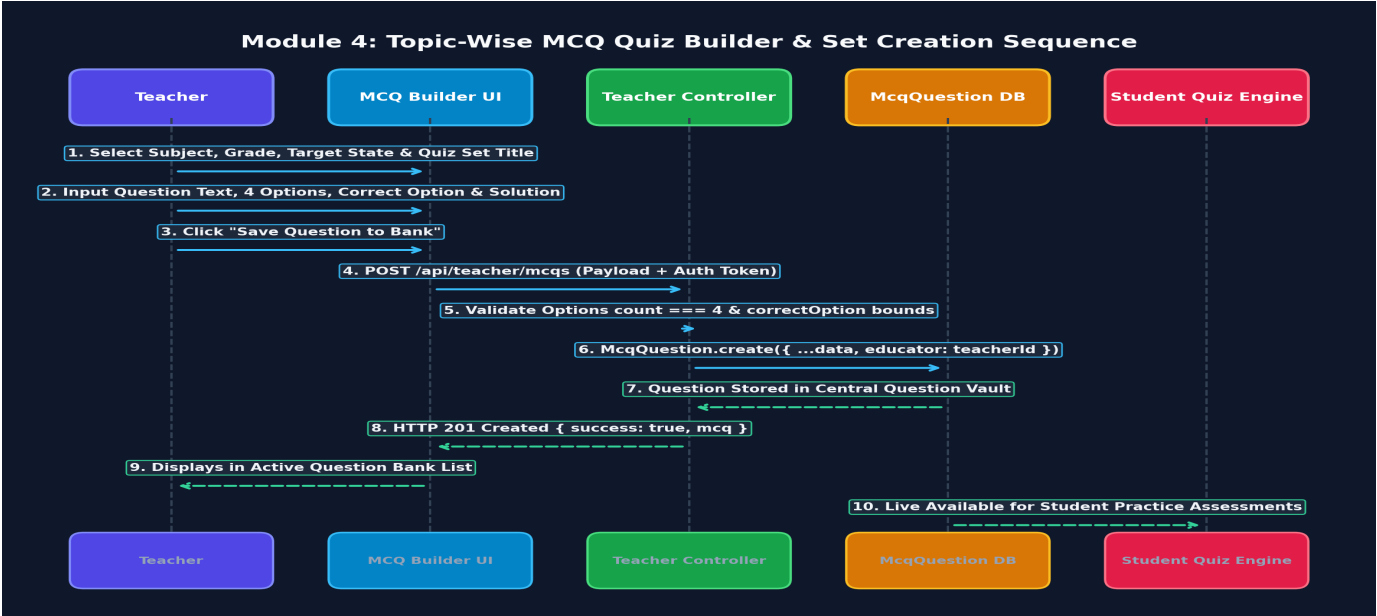
Sequence Execution Steps & Data Exchange:

#	Sender	Receiver	Action / Payload Description	DB / System Impact
1	Teacher	Classroom UI	Navigates to 'My Courses & Batches' tab	Triggers course list fetch
2	Classroom UI	TeacherCtrl	Dispatches GET /api/teacher/courses	Includes educator auth bearer token
3	TeacherCtrl	MongoDB	Executes Course.find({ instructor: teacherId }).sort({ createdAt: -1 })	Fetches educator courses
4	MongoDB	TeacherCtrl	Returns course objects with deliverable sub-documents	Indexed by instructor ID
5	TeacherCtrl	Classroom UI	Renders Course Cards with badge counters (PDFs, Tests, Lectures)	Reactive UI card render
6	Teacher	Classroom UI	Clicks 'Inspect Enrolled Students' button on specific batch	Triggers student roster query
7	Classroom UI	TeacherCtrl	Dispatches GET /api/teacher/courses/:courseId/students	Requests roster for batch
8	TeacherCtrl	MongoDB	Queries Enrollment records joining User profiles (Name, Mobile, State)	Aggregates verified enrollment list
9	MongoDB	TeacherCtrl	Returns enrolled student array with payment status	Confirms verified learners
10	TeacherCtrl	Classroom UI	Displays interactive roster table with learner details	Client renders student table
11	Teacher	Meet / Stream	Clicks '■ Start Live Class' button -> Launches Google Meet / Zoom	Redirects to external classroom

MODULE 4 SPECIFICATION

Module 4: Topic-Wise MCQ Quiz Builder & Set Creation Sequence

The MCQ Quiz Builder equips educators to author individual questions or batch import test papers. Each question encapsulates comprehensive metadata including target exam, grade level, topic categorization, four answer options, correct answer indicator, and faculty step-by-step explanatory notes.



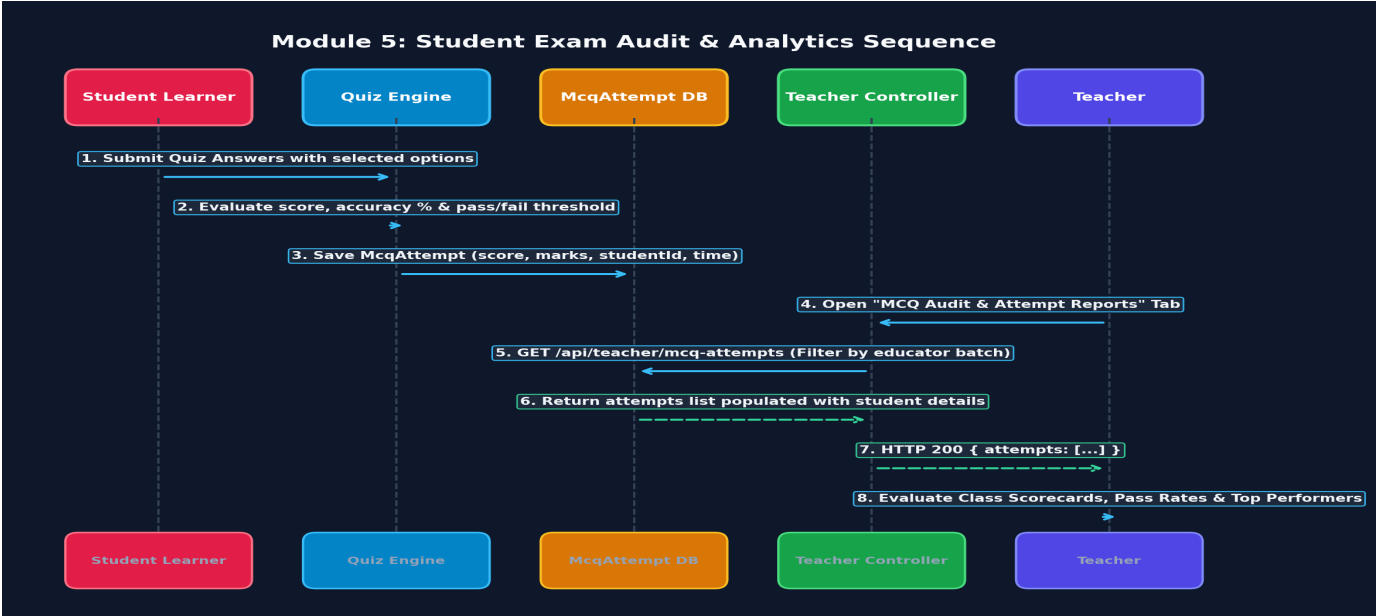
Sequence Execution Steps & Data Exchange:

#	Sender	Receiver	Action / Payload Description	DB / System Impact
1	Teacher	MCQ Builder	Selects Subject, Grade/Board, Target State & Quiz Set Title	Defines question taxonomy context
2	Teacher	MCQ Builder	Inputs Question text, 4 Options, Correct Option Index (A-D) & Solution	Client validates required inputs
3	Teacher	MCQ Builder	Clicks '■ Save Question to Bank' button	Submits question payload
4	MCQ Builder	TeacherCtrl	Dispatches POST /api/teacher/mcqs	Sends question payload over HTTPS
5	TeacherCtrl	TeacherCtrl	Validates option count === 4 and correctOption index bounds	Prevents invalid exam formatting
6	TeacherCtrl	McqQuestion DB	Executes McqQuestion.create({ ...data, educator: teacherId })	Inserts question into repository
7	McqQuestion DB	TeacherCtrl	Returns persisted question document with ID	Document committed to database
8	TeacherCtrl	MCQ Builder	Returns HTTP 201 Created { success: true, mcq }	Triggers success feedback alert
9	MCQ Builder	Teacher	Displays newly added question in 'Active Question Bank' list	Question available for editing
10	McqQuestion DB	Student Quiz	Question immediately available in Student Practice Assessment sets	Live evaluation pool updated

MODULE 5 SPECIFICATION

Module 5: Student Exam Audit & Analytics Sequence

The Exam Audit module tracks real-time student quiz performance across all authored test sets. Educators analyze student scorecards, percentage distributions, completion timestamps, and pass/fail ratios to identify curriculum bottlenecks and high-yield revision topics.



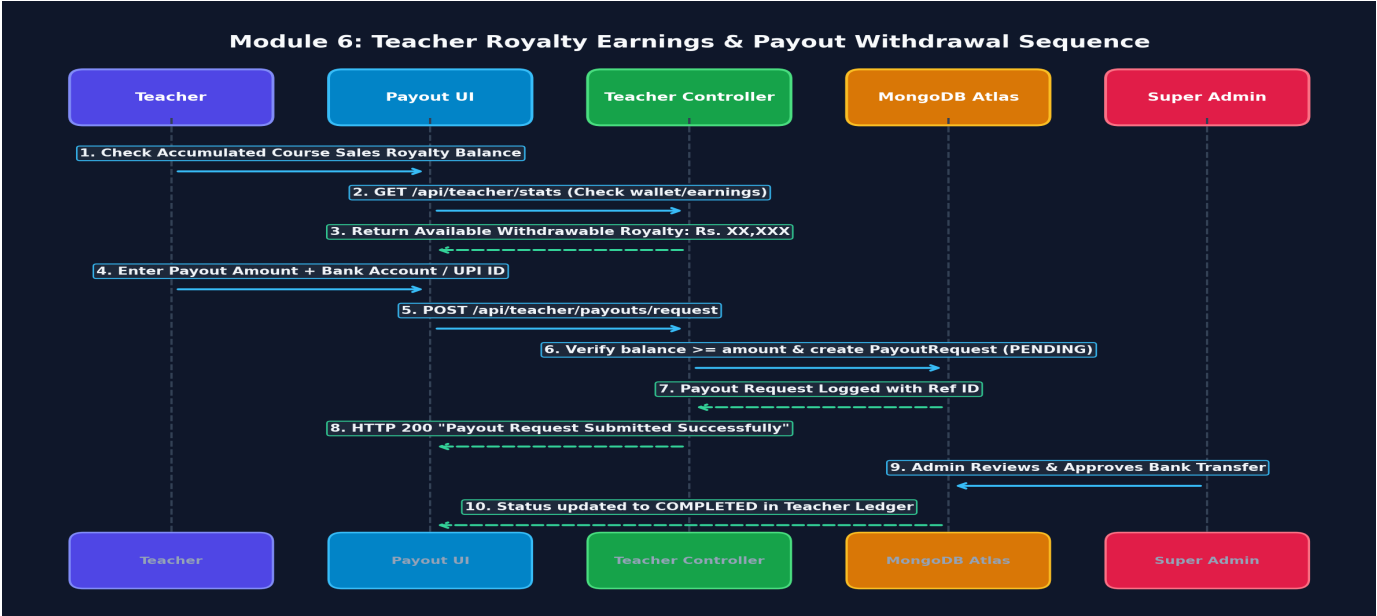
Sequence Execution Steps & Data Exchange:

#	Sender	Receiver	Action / Payload Description	DB / System Impact
1	Student	Quiz Engine	Student completes quiz assessment and submits answer sheet	Student completes assessment
2	Quiz Engine	Quiz Engine	Computes marks, percentage, passed/failed boolean & timing	Automated scoring engine
3	Quiz Engine	McqAttempt DB	Stores McqAttempt record (score, totalMarks, percentage, studentId)	Audit record logged in database
4	Teacher	TeacherCtrl	Educator opens 'MCQ Audit & Attempt Reports' tab	Triggers audit records fetch
5	TeacherCtrl	McqAttempt DB	Executes McqAttempt.find({ educatorCourses }).populate('student')	Queries student submissions
6	McqAttempt DB	TeacherCtrl	Returns attempts array with student profiles and scores	Aggregates exam performance data
7	TeacherCtrl	Teacher	Returns HTTP 200 { success: true, attempts: [...] }	Sends report data to educator UI
8	Teacher	Teacher	Educator reviews class pass percentage, top performers & struggle topics	Informs pedagogical adjustments

MODULE 6 SPECIFICATION

Module 6: Teacher Royalty Earnings & Payout Withdrawal Sequence

Educators earn automated royalties on every student course enrollment. This module provides a transparent earnings ledger, withdrawal request mechanism with bank account / UPI routing, and status tracking through Super Admin financial verification and settlement.



Sequence Execution Steps & Data Exchange:

#	Sender	Receiver	Action / Payload Description	DB / System Impact
1	Teacher	Payout UI	Inspects Accumulated Course Sales Royalty Balance	Educator checks earnings balance
2	Payout UI	TeacherCtrl	Dispatches GET /api/teacher/stats (Checks wallet & earnings ledger)	Queries royalty wallet status
3	TeacherCtrl	Payout UI	Returns Available Withdrawable Balance (e.g. ₹48,500)	Displays balance on payout panel
4	Teacher	Payout UI	Enters desired Withdrawal Amount + Bank Account / IFSC / UPI ID	Specifies payout destination
5	Payout UI	TeacherCtrl	Dispatches POST /api/teacher/payouts/request (amount, bankDetails)	Submits payout withdrawal request
6	TeacherCtrl	MongoDB	Validates balance >= amount, creates PayoutRequest document (PENDING)	Locks funds in escrow ledger
7	MongoDB	TeacherCtrl	Returns created payout record with unique Transaction Ref ID	Request queued for admin approval
8	TeacherCtrl	Payout UI	Returns HTTP 200 'Payout Request Submitted for Processing'	Displays confirmation banner
9	Super Admin	MongoDB	Admin reviews pending request in Super Admin portal & executes transfer	Disburses funds via bank/UPI
10	MongoDB	Payout UI	Payout record status transitions from 'PENDING' to 'COMPLETED'	Updates teacher ledger status